

コマ 14: グローバル変数・スコープ管理

今日のゴール

関数外で宣言されたグローバル変数を扱い、ローカル変数とグローバル変数が混在するプログラムを動かす。

第 13 回までは、変数はすべて関数内のローカル変数としてスタックフレーム上に置いていた。この回では、関数の外にある変数をプログラム全体から参照できるようにする。

```
int total;

int add(int x) {
    total = total + x;
    return total;
}
```

`total` は関数の外で宣言されているため、グローバル変数である。`add()` の中からも `main()` の中からも同じ `total` を参照する。

1 この回で扱う範囲

対象にする機能は次の通り。

種類	例
未初期化グローバル変数	<code>int count;</code>
初期値付きグローバル変数	<code>int base = 7;</code>
グローバル構造体変数	<code>Pair gp;</code>
ローカルによる隠蔽	グローバル <code>x</code> とローカル <code>x</code>
スコープ探索	local scopes → globals

この回では、グローバル変数の初期値は整数定数だけに限定する。配列や構造体の初期化リスト、文字列による初期化は扱わない。

2 ASTを確認する: グローバル変数

まず、関数外の変数宣言がASTでどう見えるか確認する。

```
python3 scaffold/parse_viewer.py sessions/14_globals_scope/tests/global_min.c
```

このプログラムの内容は次の通り。

```
int total;

int add(int x) {
    total = total + x;
    return total;
}

int main() {
    total = 0;
    add(10);
    add(20);
    return total;
}
```

実行すると、次のようなS式が表示される。

```
(program (decl "total" :type int)
  (funcdef "add" :type int
    (params (param "x" :type int))
    (block
      (exprstmt
        (assign (var "total")
          (add (var "total") (var "x")))))
      (return (var "total"))))
  (funcdef "main" :type int (params)
    (block
      (exprstmt
        (assign (var "total") (num 0)))
      (exprstmt
        (call "add"
          (args (num 10))))
      (exprstmt
        (call "add"
```

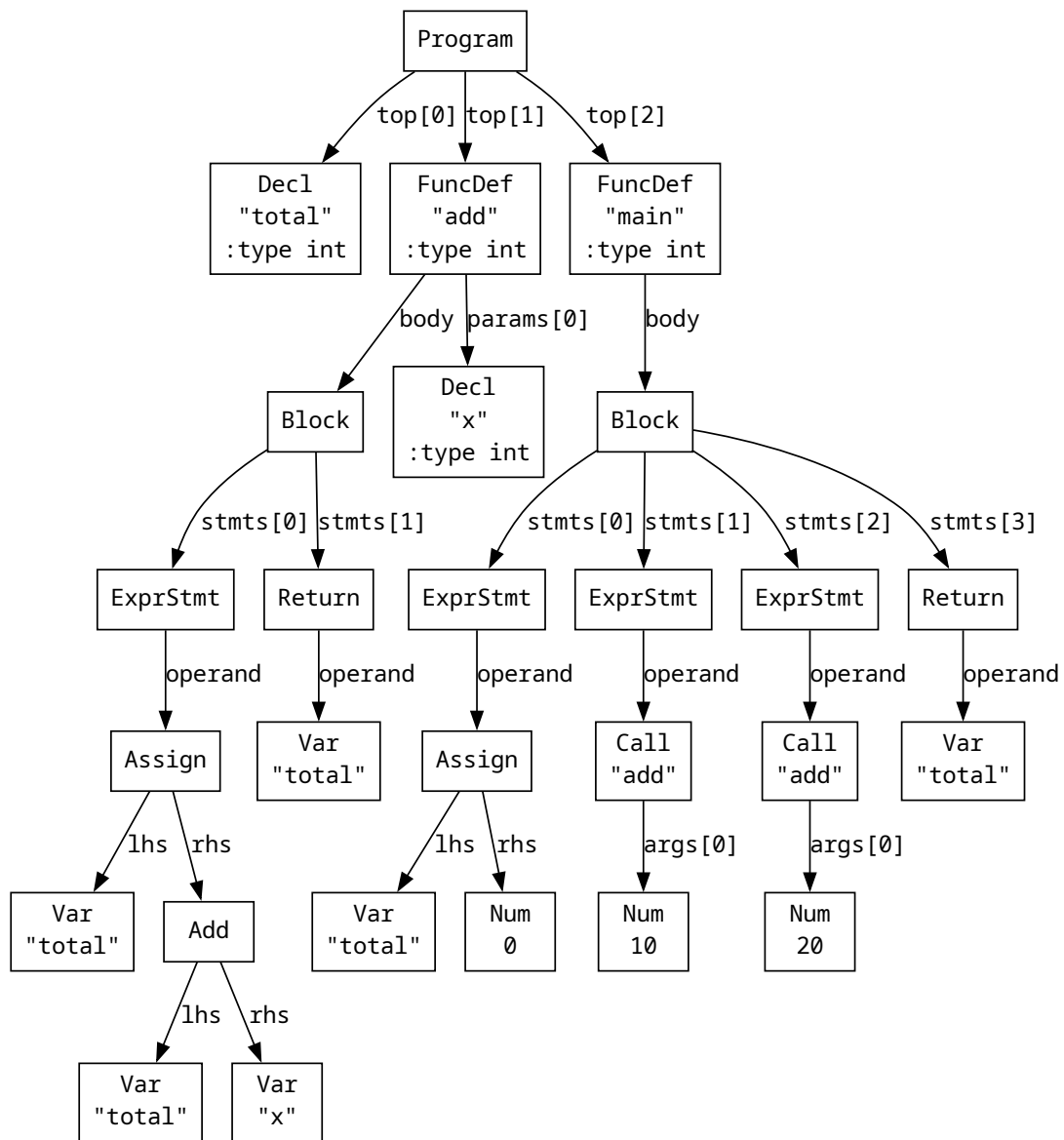


図1 global_min.c のAST

```

    (args (num 20)))
  (return (var "total")))))

```

トップレベルにある `(decl "total" :type int)` がグローバル変数宣言である。同じ `'Decl'` ノードでも、関数本体のブロック内にあればローカル変数、トップレベルにあればグローバル変数として扱う。

3 AST を確認する: ローカル変数による隠蔽

次に、同じ名前のグローバル変数とローカル変数がある例を見る。

```
python3 scaffold/parse_viewer.py sessions/14_globals_scope/tests/shadow_min.c
```

このプログラムの内容は次の通り。

```
int value;

int read_global() {
    return value;
}

int main() {
    int value;
    value = 5;
    return value + read_global();
}
```

実行すると、次のような S 式が表示される。

```
(program (decl "value" :type int)
  (funcdef "read_global" :type int (params)
    (block
      (return (var "value")))))
(funcdef "main" :type int (params)
  (block (decl "value" :type int)
    (exprstmt
      (assign (var "value") (num 5)))
    (return
      (add (var "value")
        (call "read_global" (args)))))))
```

`main()` の中の `value` はローカル変数である。一方、`read_global()` の中にはローカル変数 `value` がないので、グローバル変数 `value` を参照する。

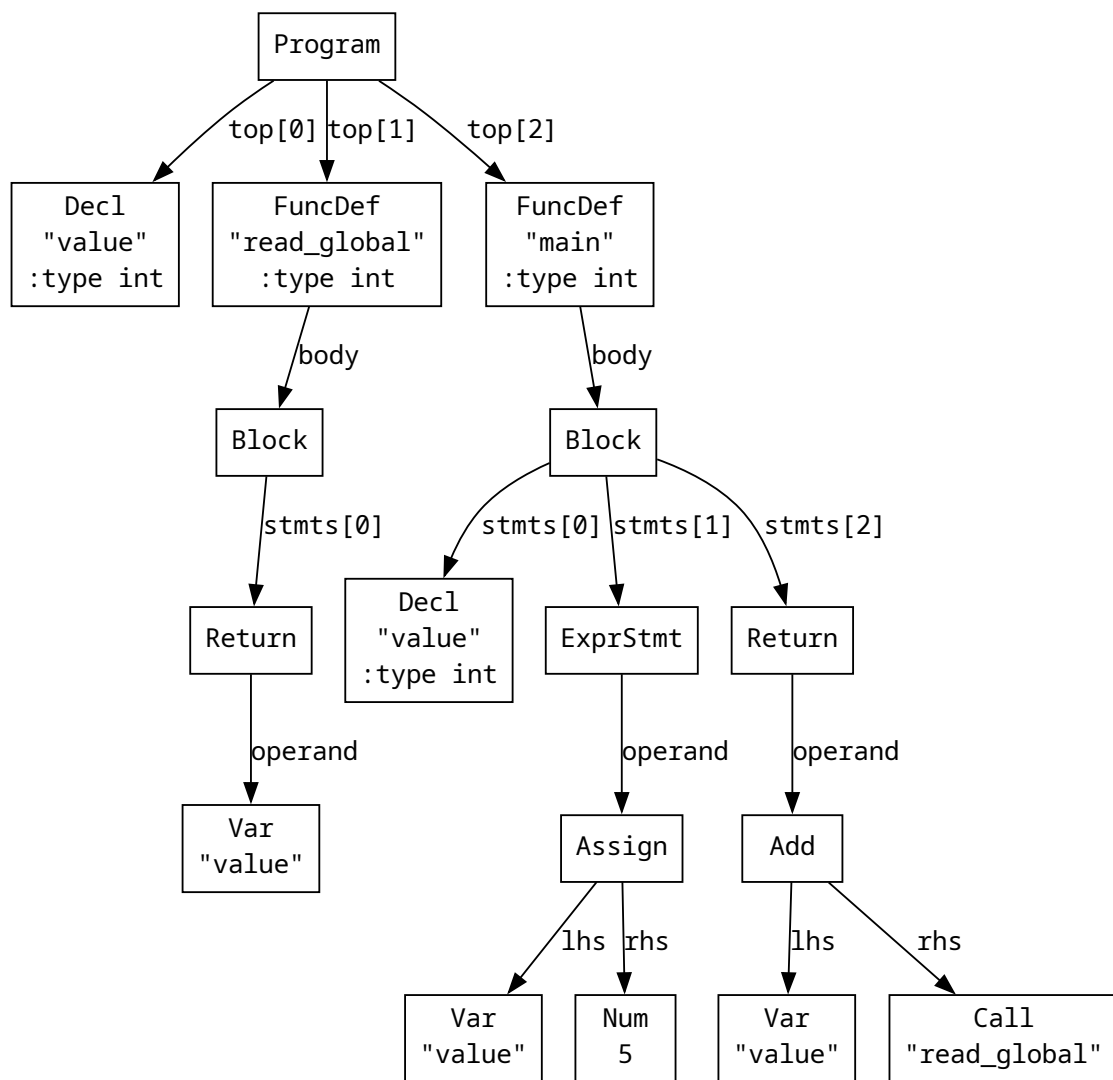


図2 shadow_min.c の AST

4 グローバル変数の置き場所

ローカル変数は関数呼び出しごとに作られるため、スタックフレームに置く。グローバル変数はプログラム全体で 1 つだけ存在し、関数呼び出しが終わっても残る。そのため、スタックではなくデータ領域に置く。

アセンブリでは、主に次の 2 つのセクションを使う。

セクション	用途
-------	----

<code>.bss</code>	未初期化、またはゼロ初期化のグローバル変数
-------------------	-----------------------

<code>.data</code>	初期値を持つグローバル変数、文字列リテラル
--------------------	-----------------------

たとえば、次の宣言を考える。

```
int count;  
int base = 7;
```

この回の実装では、おおよそ次のように出力する。

```
.data  
.globl base  
base:  
.word 7  
  
.bss  
.globl count  
count:  
.zero 8
```

`.zero 8` は 8 バイトぶんのゼロ領域を確保する指定である。`int` は 4 バイトだが、この教材では単純化のためグローバル領域も 8 バイト境界に揃えて確保する。

ローカル変数はスタック、`malloc` で確保した領域はヒープ、グローバル変数は `.data` / `.bss` に置かれる。どの領域に置くかが決まると、アドレスの計算方法（`s0` からのオフセットか、ラベルか）も決まる。

5 グローバル変数のアドレス

ローカル変数のアドレスは、これまで通り `s0` からのオフセットで計算する。

```
addi a0, s0, -24
```

グローバル変数にはアセンブリ上のラベルを付ける。そのため、アドレスは `la` で取得できる。

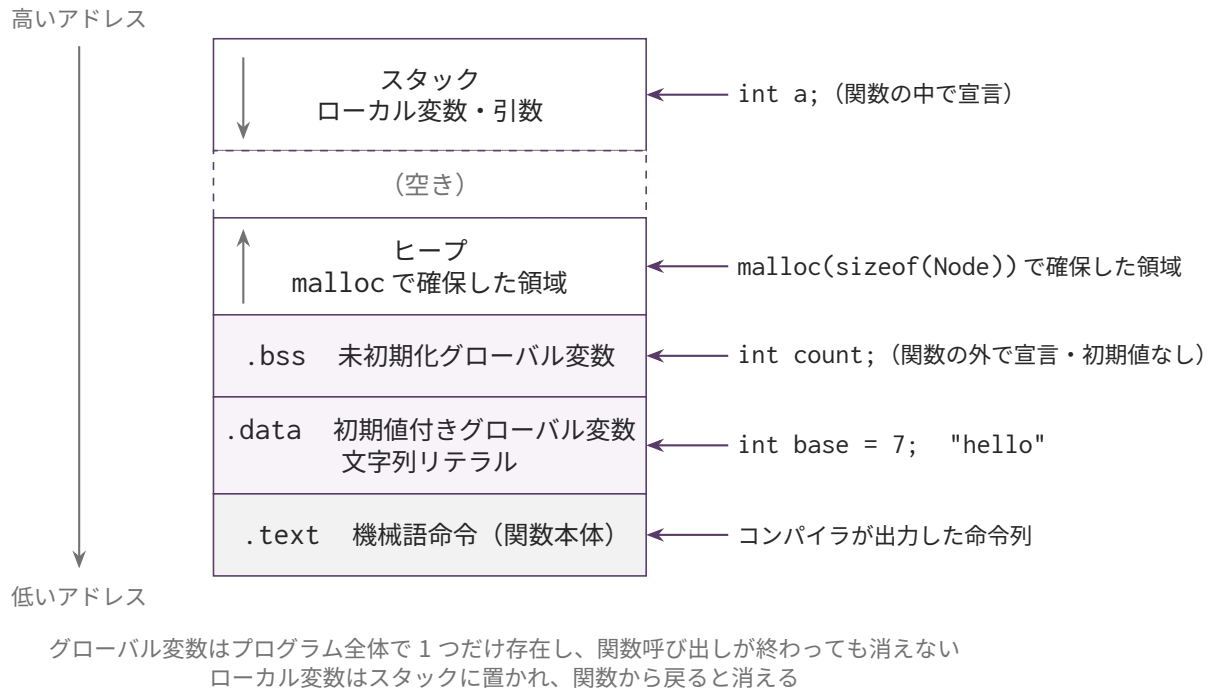


図 3 プログラム実行時のメモリ全体像と変数の置き場所

```
la a0, total
```

したがって `self.codegen_lval_Var(node)` では、`self._is_local()` を使って変数がローカルかグローバルかで分岐する。

```
def codegen_lval_Var(self, node):
    if self._is_local(node.name):
        off = self._locals[node.name][0]
        self.emit(f" addi a0, s0, {off}")
    else:
        self.emit(f" la a0, {node.name}")
```

値を読む処理は同じである。まず lvalue としてアドレスを求め、その型に応じて `lw`、`lb`、`ld` でロードする。

6 変数表を 2 種類に分ける

第 13 回までは `symbols` という辞書にローカル変数だけを入れていた。第 14 回では、グローバル変数とローカル変数に分ける。

```
# インスタンス変数として持つ
```

```
self._globals: dict[str, tuple[str, int | None]] # name → (ty_str, init_value_or_...
self._locals: dict[str, tuple[int, str]]         # name → (offset, ty_str)
```

`self._globals` はプログラム全体で 1 つだけ持つ。トップレベルの宣言を `self.collect_globals(p` で集めてからコード生成に入る。

`self._locals` は関数ごとに 1 つ持ち、`self._reset_func_state()` で関数に入るたびに空にする。第 13 回までのブロックスコープの `push/pop` のリスト方式より単純で、1 関数につき 1 つの辞書で済む。

変数名を探すときは、まず `self._locals` を調べ、なければ `self._globals` を調べる。

```
def lookup_var(self, name, line):
    if name in self._locals:
        return ('local', self._locals[name])
    if name in self._globals:
        return ('global', self._globals[name])
    raise self._error_at_node_line(f"未定義の変数: {name}", line)
```

この順序にすることで、ローカル変数が同名のグローバル変数を隠す動作を実現できる。

7 ブロックスコープ

C では、ブロックごとにスコープが作られる。

```
int x;

int main() {
    int x;
    x = 5;
    return x;
}
```

この `main()` の中の `x` は、グローバル変数 `x` とは別の変数である。`main()` の中で `x` と書いたときは、内側のローカル変数を優先して参照する。

スキャフォールドでは関数ごとに 1 つの `self._locals` 辞書を持つ。同じ名前の変数が既に `self._locals` にあっても、新しい宣言なら上書きせずエラーにする（または新しいオフ

セットで登録する)。変数探索は常に `self._locals` → `self._globals` の順なので、ローカルがグローバルを隠す動作が自然に実現される。

ブロック内の宣言のオフセット割り当ては、関数本体を走査して各ローカル変数のオフセットを先に決めておく。これは第 13 回までのブロックを `push/pop` する方式より単純である。

8 実装手順

1. 第 13 回の実装を `sessions/14_globals_scope/mycc.py` に反映する（スケルトンの `importlib` 継承により、前回の `Codegen` クラスを継承する。新機能の `handler` だけを実装すればよい。）
2. `self._globals` と `self._locals` を追加する（スケルトンにあらかじめ書かれている）
3. トップレベルの `'Decl'` ノードを `self.collect_globals()` で集める
4. 未初期化グローバル変数を `.bss` に出力する
5. 初期値付きグローバル変数を `.data` に出力する
6. `self.lookup_var()` を `self._locals` → `self._globals` の順にする
7. `self.codegen_lval_Var()` で `self._is_local()` を使って分岐し、グローバル変数なら `la a0, name` を出す
8. `global_counter.c`、`global_local_shadow.c`、`global_struct.c` を通す

9 編集するファイル

`sessions/14_globals_scope/` 以下のスケルトンファイルを編集する。

ファイル	実装するハンドラ・メソッド
<code>mycc.py</code>	<code>Codegen14</code> クラス。 <code>collect_globals()</code> 、 <code>lookup_var()</code> 、 <code>codegen_lval_Var()</code> 、 <code>_is_local()</code> 、 <code>_emit_global_data()</code> など
(AST パーサ)	変更不要（ <code>'Decl'</code> ノードは第 13 回から存在する）

10 テスト

```
python3 scaffold/test_runner.py sessions/14_globals_scope
```

この回の主要テストは次の通り。

テスト	内容	期待値
global_counter.c	グローバル call_count / total と printf	stdout 60, exit 3
global_init.c	初期値付きグローバル 変数	12
global_local_shadow.c	ローカル変数がグローバ ル変数を隠す	5
global_struct.c	グローバル構造体変数と . / &	30